Question 1

Part A

I. Some assumptions that I would make are:
   A. Since this handwritten process doesn't include a primary identifier for each order, I need to create a composite primary key. This primary key will include customerNo + orderDate + orderTime. This assumes that a customer cannot place two orders at the same date and time.
   B. partNo determines partName, partType, cageCode, and unitPrice
   C. customerNo determines customerName and customerType
   D. customerNo + orderDate + orderTime + partNo determines quantityOrdered
   E. customerNo + orderDate + orderTime determines employeeName (this assumes one employee handles each order and that there are no repeating employee names in the company)

Part B, C, D

0NF (Current)

part_order
customerName
customerNo
customerType
orderDate
orderTime
employeeName
partNumber
partNo
partName
partType
cageCode
quantityOrdered
unitPrice

- Current status is 0NF because there are no defined primary keys, and the current slip of paper is not in a conventional ERD table-format.

1NF

part_order	
PK	**customerNo**
PK	**orderDate**
PK	**orderTime**
	customerNo
	customerType
	employeeName
	partNumber
	partNo
	partName
	partType
	cageCode
	quantityOrdered
	unitPrice

- To get to 1NF from 0NF, I created clearly defined composite primary keys

2NF & 3NF

customer	
PK FK	***customerNo***
	customerName
	customerType

part_order	
PK FK	***customerNo***
PK FK	***orderDate***
PK FK	***orderTime***
	employeeName

part	
PK FK	***partNo***
	partType
	partName
	cageCode
	unitPrice

part_orderline	
PK	**customerNo**
PK	**orderDate**
PK	**orderTime**
PK	**partNo**
	quantityOrdered

- From 1NF to 2NF, need to remove any partial dependencies. Creating separate entities:
- Customer had to have their own entity because customerName and customerType depend on customerNo only. Similar to the part information (type, name, price, cage).
- part_orderline is the main order table, hence quantityOrdered is unqiue to each order
- Since there are no transitive dependencies, this 2NF will automatically become a 3NF

Question 2

Part A

I. Some assumptions that I would make are:
   A. Composite primary key of staffNo + appointmentDate + appointmentTime. This assumes that a therapist can only see one patient at a time.
   B. staffNo + appointmentDate + appointmentTime determines patNo
   C. staffNo + appointmentDate determines branchNo (because a staff can at one branch on a given day)
   D. staffNo determines therapistName
   E. patNo determines patName

Part B, C, D

0NF (Current)

appointment
staffNo
therapistName
patNo
patName
appointmentDate
appointmentTime
branchNo

- Currently, the table is in 0NF because there are now clearly defined primary keys. However, there are no repeating attributes/rows

1NF

appointment	
PK	**staffNo**
PK	**appointmentDate**
PK	**appointmentTime**
	patName
	therpaistName
	patNo
	branchNo

- To transform it from 0NF to 1NF, created clearly defined composite primary key. I have chosen those three because it follows my assumption that a staff member can only provide one therapy session at a time

2NF

appointment	
PK	**staffNo**
PK	**appointmentDate**
PK	**appointmentTime**
	patNo
	patName

staff	
PK FK	***staffNo***
	therpaistName

staff_schedule	
PK FK	***staffNo***
PK FK	***appointmentDate***
	branchNo

- To transfrom it from 1NF to 2NF, need to remove the partial dependencies such as staffNo and staffName. staff_schedule shows that a staff member can work at multiple branches depending on the date

3NF

appointment	
PK	**staffNo**
PK	**appointmentDate**
PK	**appointmentTime**
	patNo

staff	
PK FK	***staffNo***
	therpaistName

patient	
PK FK	***patNo***
	patName

staff_schedule	
PK FK	***staffNo***
PK FK	***appointmentDate***
	branchNo

- To transfrom it from 2NF to 3NF, need to remove transitive dependencies which was patName depending on patNo, which was depending on the primary key. So I created a separate patient table

Question 3

Part A

I. Some assumptions that I would make are:
   A. eNo determines eName
   B. eventNo determines eventLoc (each event has one location)
   C. eNo + contractNo (is the primary composite key) also determines hours and eventNo. eNo + contractNo assumes that an employee's hours on a contract are recorded as one total.
   D. contractNo determines eventNo (because each contract is tied to only one event, but an event can have multiple contracts)

Part B, C, D

0NF (Current)

event
eNo
contractNo
hours
eName
eventNo
eventLoc

- Currently in 0NF because no clearly identified primary key However, no repeating attributes or rows of information.

1NF

event	
PK	**eNo**
PK	**contractNo**
	hours
	eName
	eventNo
	eventLoc

- To transform it from 0NF to 1NF, need to clearly identify primary key (composite in this case of eNo and contractNo)

2NF

employee_contract	
PK	***eNo***
PK	***contractNo***
	hours

employee	
PK	**eNo**
	eName

contract	
PK	**contractNo**
	eventNo
	eventLoc

- To transform from 1NF to 2NF, need to remove partial dependencies. Since eName only depends on eNo and eventNo & eventLoc only depend on contractNo, I created two separate entities

3NF

employee_contract	
PK	***eNo***
PK	***contractNo***
	hours

employee	
PK FK	***eNo***
	eName

contract	
PK FK	***contractNo***
	eventNo

event	
PK FK	***eventNo***
	eventLoc

- To make it into 3NF, need to deal with transitive dependency: that is, eventLoc depends on eventNo, which depends on contractNo, so created a separate event entity.